Generalized Linear Mixed Models (GLMM)

Summary

This page consists of the following parts:

1. This part explores different GLMM estimation methods (Gaussian–Hermite Quadrature (GHQ), Laplace approximation, or Penalized Quasi-Likelihood (PQL)) for a binary data example.
2. This part provides a guide for translating common SAS `PROC GLIMMIX` model specifications into R, with 10 examples.

1. Comparing Different GLMM Estimation Methods for Binary Data Example

Overview

Differences between SAS and R, (and even between different functions/procedures within same software) can lead to different results. These differences do not imply that one tool is more reliable than another but rather reflect variations in default settings and computational methods.

The following section summarizes the comparisons conducted using the dataset from "Gee Model for Binary Data" in the SAS/STAT Sample Program Library [1]. Each subsection corresponds to a different GLMM estimation method: **Gaussian–Hermite Quadrature (GHQ), Laplace approximation, or Penalized Quasi-Likelihood (PQL)**, and outlines the specific options and functions required to obtain consistent or comparable results across software platforms.

In general, different R functions were able to reproduce SAS GLMM results (using either GHQ or the Laplace approximation) when applying naïve or sandwich standard errors and assuming infinite degrees of freedom (ddff). However, only approximate agreement was achieved when comparing estimates based on the Between-Within ddf. When using PQL, additional discrepancies emerged due to differences in the underlying estimation methods across software (i.e., PQL in R versus RSPL in SAS).

Gauss-Hermite Quadrature (GHQ) approximation

S.E.	Ddff	SAS	R	Match?
Naive	Inf	GLIMMIX with ddff=none	lme4::glmer	Yes
Naive	Inf	GLIMMIX with ddff=none	GLMMadaptive::mixed_model	Aprox.
Naive	BW	GLIMMIX with ddff=BW (default)	lme4::glmer & parameters::dof_betwithin [1]	Aprox [(1)]
Sandwich	Inf	GLIMMIX with empirical option & ddff=none	lme4::glmer & merDeriv::sandwich	Aprox [(2)]
Sandwich	BW	GLIMMIX with empirical option & ddff=BW(default)	lme4::glmer, merDeriv::sandwich & parameters::dof_betwithin [1]	Aprox [(1)]

BW: Between-Within, Inf: Infinite.

1. In R, the function `parameters::dof_betwithin` uses a heuristic implementation of the between-within method and therefore does not reproduce the exact results reported in Li and Redden (2015); instead, it yields values that are similar but not identical. In contrast, SAS computes the exact between-within degrees of freedom.
2. As `glmer` does not provide cluster-level scores, the `sandwich` function cannot compute clustered robust estimator, so it does not correspond to SAS's emprical covariance [2].

Laplace approximation

S.E.	Ddff	SAS	R	Match?
Naive	Inf	GLIMMIX with ddff=none	lme4::glmer	Aprox. (2)
Naive	Inf	GLIMMIX with ddff=none	glmmTMB::glmmTMB	Yes
Naive	BW	GLIMMIX with ddff=BW (default)	lglmmTMB::glmmTMB & parameters::dof_betwithin [1]	Aprox. (1)
Sandwich	Inf	GLIMMIX with empirical option & ddff=none	glmmTMB::glmmTMB & clubSandwich::vcovHC	Yes
Sandwich	BW	GLIMMIX with empirical option & ddff=BW(default)	lme4::glmer, merDeriv::sandwich & parameters::dof_betwithin [1]	Aprox (1)

BW: Between-Within, Inf: Infinite.

(1): In R, the function `parameters::dof_betwithin` uses a heuristic implementation of the between-within method and therefore does not reproduce the exact results reported in Li and Redden (2015); instead, it yields values that are similar but not identical. In contrast, SAS computes the exact between-within degrees of freedom.

(2): Although `lme4::glmer` produced estimates consistent with SAS when using GHQ, it exhibited convergence issues that were not present in either SAS or `glmmTMB::glmmTMB`. After adjusting the optimization settings (see R section), only approximate agreement with the SAS results was achieved.

PQL

S.E.	Ddff	SAS	R	Match?
Naive	Residual	GLIMMIX with method=RSPL & ddfm=residual	MASS::glmmPQL	Aprox. (1)

(1): Results differ because of the different computation methods (PQL vs. RSPL)

GHQ Details

In this section, SAS and R results obtained under the GHQ approach are compared, after adjusting the necessary settings or applying additional functions to ensure alignment across software (See summary table above and respective R and SAS sections).

Model Based SE and Infinite ddff

Similar results in the estimate were obtained in SAS and R (using `lme4::glmer`), and approximated results were obtained with mixed_model (See R section for difference details between R functions).

SAS

Effect	trtp	avisitn	Estimate	StdErr	DF	tValue	Probt
Intercept		_	-0.06170	.5274	1	-0.12	0.9069
trtp	A	_	1.57500	.7835	1	2.01	0.0444
avisitn		2	-0.87191	.5476	1	-1.59	0.1113
avisitn		3	-0.28727	.5369	1	-0.54	0.5926
avisitn		4	-0.43174	.5386	1	-0.80	0.4228
trtp*avisitn	A	2	1.04384	.8036	1	1.30	0.1939
trtp*avisitn	A	3	0.63608	.8000	1	0.80	0.4265
trtp*avisitn	A	4	-0.22001	.7866	1	-0.28	0.7797

R

glmer

	Estimate	Std. Error	z value	Pr(>|z|)
(Intercept)	-0.0617	0.5274	-0.12	0.907
trtpA	1.5750	0.7835	2.01	0.044
avisitn2	-0.8720	0.5476	-1.59	0.111
avisitn3	-0.2873	0.5369	-0.54	0.593
avisitn4	-0.4318	0.5386	-0.80	0.423
trtpA:avisitn2	1.0440	0.8036	1.30	0.194
trtpA:avisitn3	0.6362	0.8000	0.80	0.426
trtpA:avisitn4	-0.2200	0.7866	-0.28	0.780

mixed_model

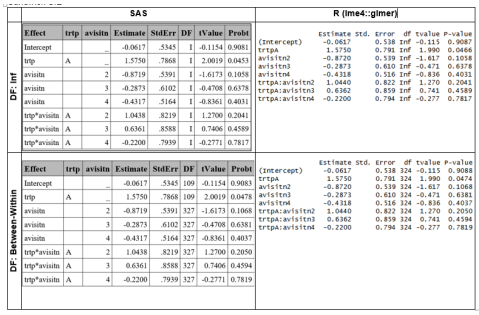
	Estimate	Std.Err	z-value	p-value
(Intercept)	-0.076	0.581	-0.131	0.90
trtpa	1.639	0.873	1.878	0.06
avisitn2	-0.875	0.547	-1.597	0.11
avisitn3	-0.289	0.537	-0.538	0.59
avisitn4	-0.433	0.539	-0.804	0.42
trtpa:avisitn2	1.046	0.803	1.303	0.19
trtpa:avisitn3	0.637	0.800	0.797	0.43
trtpa:avisitn4	-0.219	0.787	-0.278	0.78

Sandwich SE

Sandwich standard errors were computed using the `EMPIRICAL` option in SAS and the `merDeriv::sandwich` function in R (See corresponding SAS and R sections for details). This approach yielded comparable,though not identical results. The slight discrepancies arise because the base

`sandwich` method cannot compute a fully cluster-robust estimator for `glmer` models, and therefore does not reproduce the exact empirical covariance structure used by SAS [2].

Between-Within ddff were obtained in R using `parameters::dof_betwithin` function, which computes an approximation, instead of the exact dff as in SAS.



SAS

DF: Inf

Effect	trtp	avisitn	Estimate	StdErr	DF	tValue	Probt
Intercept		_	-0.0617	.5345	1	-0.1154	0.9081
trtp	A	_	1.5750	.7868	1	2.0019	0.0453
avisitn		2	-0.8719	.5391	1	-1.6173	0.1058
avisitn		3	-0.2873	.6102	1	-0.4708	0.6378
avisitn		4	-0.4317	.5164	1	-0.8361	0.4031
trtp*avisitn	A	2	1.0438	.8219	1	1.2700	0.2041
trtp*avisitn	A	3	0.6361	.8588	1	0.7406	0.4589
trtp*avisitn	A	4	-0.2200	.7939	1	-0.2771	0.7817

DF: Between-Within

Effect	trtp	avisitn	Estimate	StdErr	DF	tValue	Probt
Intercept		_	-0.0617	.5345	109	-0.1154	0.9083
trtp	A	_	1.5750	.7868	109	2.0019	0.0478
avisitn		2	-0.8719	.5391	327	-1.6173	0.1068
avisitn		3	-0.2873	.6102	327	-0.4708	0.6381
avisitn		4	-0.4317	.5164	327	-0.8361	0.4037
trtp*avisitn	A	2	1.0438	.8219	327	1.2700	0.2050
trtp*avisitn	A	3	0.6361	.8588	327	0.7406	0.4594
trtp*avisitn	A	4	-0.2200	.7939	327	-0.2771	0.7819

R (lme4::glmer)

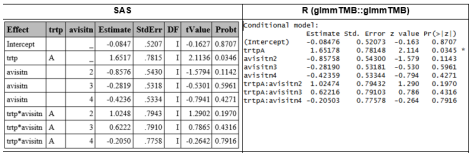
```
             Estimate Std. Error  df t value P-value
(Intercept)   -0.0617     0.538 Inf  -0.115  0.9087
trtpA          1.5750     0.791 Inf   1.990  0.0466
avisitn2      -0.8720     0.539 Inf  -1.617  0.1058
avisitn3      -0.2873     0.610 Inf  -0.471  0.6378
avisitn4      -0.4318     0.516 Inf  -0.836  0.4031
trtpA:avisitn2  1.0440     0.822 Inf   1.270  0.2041
trtpA:avisitn3  0.6362     0.859 Inf   0.741  0.4589
trtpA:avisitn4 -0.2200     0.794 Inf  -0.277  0.7817
```

```
             Estimate Std. Error  df t value P-value
(Intercept)   -0.0617     0.538 324  -0.115  0.9088
trtpA          1.5750     0.791 324   1.990  0.0474
avisitn2      -0.8720     0.539 324  -1.617  0.1068
avisitn3      -0.2873     0.610 324  -0.471  0.6381
avisitn4      -0.4318     0.516 324  -0.836  0.4037
trtpA:avisitn2  1.0440     0.822 324   1.270  0.2050
trtpA:avisitn3  0.6362     0.859 324   0.741  0.4594
trtpA:avisitn4 -0.2200     0.794 324  -0.277  0.7819
```

Laplace Details

Model Based SE and Infinite ddff

Results obtained with SAS and `glmmTMB::glmmTMB` matched, with minor differences in the latest decimal, attributable to different rounding between software.



SAS

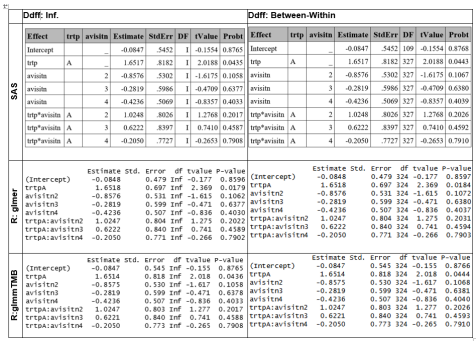
Effect	trtp	avisitn	Estimate	StdErr	DF	tValue	Probt
Intercept		_	-0.0847	.5207	1	-0.1627	0.8707
trtp	A	_	1.6517	.7815	1	2.1136	0.0346
avisitn		2	-0.8576	.5430	1	-1.5794	0.1142
avisitn		3	-0.2819	.5318	1	-0.5301	0.5961
avisitn		4	-0.4236	.5334	1	-0.7941	0.4271
trtp*avisitn	A	2	1.0248	.7943	1	1.2902	0.1970
trtp*avisitn	A	3	0.6222	.7910	1	0.7865	0.4316
trtp*avisitn	A	4	-0.2050	.7758	1	-0.2642	0.7916

R (glmmTMB::glmmTMB)

```
Conditional model:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.08476    0.52073  -0.163   0.8707
trtpA         1.65178    0.78148   2.114   0.0345 *
avisitn2     -0.85758    0.54300  -1.579   0.1143
avisitn3     -0.28190    0.53181  -0.530   0.5961
avisitn4     -0.42359    0.53344  -0.794   0.4271
trtpA:avisitn2  1.02474    0.79432   1.290   0.1970
trtpA:avisitn3  0.62216    0.79103   0.786   0.4316
trtpA:avisitn4 -0.20503    0.77578  -0.264   0.7916
```

Sandwich SE

Sandwich SE were obtained using `clubSandwich::vovHC` R function, obtainng similar SE to SAS. However, there are differences in the between-within ddff, because the R function is based on a heuristic.



Ddff: Inf.

SAS

Effect	trtp	avisitn	Estimate	StdErr	DF	tValue	Probt
Intercept		_	-0.0847	.5452	1	-0.1554	0.8765
trtp	A	_	1.6517	.8182	1	2.0188	0.0435
avisitn		2	-0.8576	.5302	1	-1.6175	0.1058
avisitn		3	-0.2819	.5986	1	-0.4709	0.6377
avisitn		4	-0.4236	.5069	1	-0.8357	0.4033
trtp*avisitn	A	2	1.0248	.8026	1	1.2768	0.2017
trtp*avisitn	A	3	0.6222	.8397	1	0.7410	0.4587
trtp*avisitn	A	4	-0.2050	.7727	1	-0.2653	0.7908

R: glmer

```
             Estimate Std. Error  df t value P-value
(Intercept)   -0.0848     0.479 Inf  -0.177  0.8596
trtpA          1.6518     0.697 Inf   2.369  0.0179
avisitn2      -0.8576     0.531 Inf  -1.615  0.1062
avisitn3      -0.2819     0.599 Inf  -0.471  0.6377
avisitn4      -0.4236     0.507 Inf  -0.836  0.4030
trtpA:avisitn2  1.0247     0.804 Inf   1.275  0.2022
trtpA:avisitn3  0.6222     0.840 Inf   0.741  0.4589
trtpA:avisitn4 -0.2050     0.771 Inf  -0.266  0.7902
```

R: glmmTMB

```
             Estimate Std. Error  df t value P-value
(Intercept)   -0.0847     0.545 Inf  -0.155  0.8765
trtpA          1.6514     0.818 Inf   2.018  0.0436
avisitn2      -0.8575     0.530 Inf  -1.617  0.1058
avisitn3      -0.2819     0.599 Inf  -0.471  0.6378
avisitn4      -0.4236     0.507 Inf  -0.836  0.4033
trtpA:avisitn2  1.0247     0.803 Inf   1.277  0.2017
trtpA:avisitn3  0.6221     0.840 Inf   0.741  0.4588
trtpA:avisitn4 -0.2050     0.773 Inf  -0.265  0.7908
```

Ddff: Between-Within

SAS

Effect	trtp	avisitn	Estimate	StdErr	DF	tValue	Probt
Intercept		_	-0.0847	.5452	109	-0.1554	0.8768
trtp	A	_	1.6517	.8182	327	2.0188	0.0443
avisitn		2	-0.8576	.5302	327	-1.6175	0.1067
avisitn		3	-0.2819	.5986	327	-0.4709	0.6380
avisitn		4	-0.4236	.5069	327	-0.8357	0.4039
trtp*avisitn	A	2	1.0248	.8026	327	1.2768	0.2026
trtp*avisitn	A	3	0.6222	.8397	327	0.7410	0.4592
trtp*avisitn	A	4	-0.2050	.7727	327	-0.2653	0.7910

R: glmer

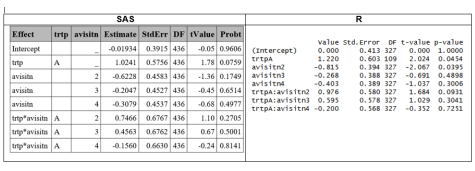
```
             Estimate Std. Error  df t value P-value
(Intercept)   -0.0848     0.479 324  -0.177  0.8597
trtpA          1.6518     0.697 324   2.369  0.0184
avisitn2      -0.8576     0.531 324  -1.615  0.1072
avisitn3      -0.2819     0.599 324  -0.471  0.6380
avisitn4      -0.4236     0.507 324  -0.836  0.4037
trtpA:avisitn2  1.0247     0.804 324   1.275  0.2031
trtpA:avisitn3  0.6222     0.840 324   0.741  0.4594
trtpA:avisitn4 -0.2050     0.771 324  -0.266  0.7903
```

R: glmmTMB

```
             Estimate Std. Error  df t value P-value
(Intercept)   -0.0847     0.545 324  -0.155  0.8766
trtpA          1.6514     0.818 324   2.018  0.0444
avisitn2      -0.8575     0.530 324  -1.617  0.1068
avisitn3      -0.2819     0.599 324  -0.471  0.6381
avisitn4      -0.4236     0.507 324  -0.836  0.4040
trtpA:avisitn2  1.0247     0.803 324   1.277  0.2026
trtpA:avisitn3  0.6221     0.840 324   0.741  0.4593
trtpA:avisitn4 -0.2050     0.773 324  -0.265  0.7910
```

PQL Details

The PQL approach uses linear approximations instead of likelihood, making it less accurate for binary outcomes compared to the GHQ or Laplace methods described above. In SAS, this is implemented by default using the Residual Pseudo-Likelihood method (method=RSPL), which is a refinement of PQL which incorporates residual adjustments to better approximate the marginal likelihood, in the GLIMMIX procedure. In R the PQL computation can be obtained using the glmmPQL function form the MASS package.



SAS

Effect	trtp	avisitn	Estimate	StdErr	DF	tValue	Probt
Intercept		_	-0.01934	0.3915	436	-0.05	0.9606
trtp	A	_	1.0241	0.5756	436	1.78	0.0759
avisitn		2	-0.6228	0.4583	436	-1.36	0.1749
avisitn		3	-0.2047	0.4527	436	-0.45	0.6514
avisitn		4	-0.3079	0.4537	436	-0.68	0.4977
trtp*avisitn	A	2	0.7466	0.6767	436	1.10	0.2705
trtp*avisitn	A	3	0.4563	0.6762	436	0.67	0.5001
trtp*avisitn	A	4	-0.1560	0.6630	436	-0.24	0.8141

R

```
             value Std.Error  DF t-value p-value
(Intercept) 0.000     0.413 327   0.000  1.0000
trtpA       1.220     0.603 109   2.024  0.0454
avisitn2   -0.815     0.394 327  -2.067  0.0395
avisitn3   -0.268     0.388 327  -0.691  0.4898
avisitn4   -0.403     0.389 327  -1.037  0.3006
trtpA:avisitn2  0.976     0.580 327   1.684  0.0931
trtpA:avisitn3  0.595     0.578 327   1.029  0.3041
trtpA:avisitn4 -0.200     0.568 327  -0.352  0.7251
```

Results differ because of the different computation methods (PQL vs. RSPL) across software. Since glmmPQL is widely recognized as less reliable for binary outcomes, more robust approaches such as Laplace or GHQ are generally preferred. Consequently, further investigation using glmmPQL is not pursued.

Outcomes with more than 2 Categories

Ordinal variables

A GLMM with ordinal outcome is fit in SAS using PROC CLIMMIX and function `ordinal::clmm,` (See corresponding SAS and R section for details), obtaining some differences in the results, because `clmm` applies different optimization algorithms (Newton, BFGS, NR methods) and its own parameterization of random effects [2].

SAS

Effect	trtp	avisitn	Estimate	StdErr	DF	tValue	Probt
Intercept		_	-0.6643	0.2596	107	-2.56	0.0119
Intercept		_	0.5638	0.2760	107	2.04	0.0435
trtp	A	_	0.4371	0.3468	107	1.26	0.2103
avisitn		2	-0.01266	0.3865	327	-0.03	0.9739
avisitn		3	0.1096	0.3403	327	0.32	0.7476
avisitn		4	-0.1714	0.3463	327	-0.49	0.6210
trtp*avisitn	A	2	-0.2617	0.5247	327	-0.50	0.6183
trtp*avisitn	A	3	-0.5723	0.4863	327	-1.18	0.2401
trtp*avisitn	A	4	-0.3095	0.4590	327	-0.67	0.5006
base		_	0.2120	0.1961	107	1.08	0.2821

R

```
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
trtpA         -0.43273    0.36061  -1.200    0.230
avisitn2       0.01155    0.35236   0.033    0.974
avisitn3      -0.10701    0.35079  -0.305    0.760
avisitn4       0.17113    0.34838   0.491    0.623
trtpA:avisitn2 0.26151    0.50072   0.522    0.601
trtpA:avisitn3 0.57354    0.50390   1.138    0.255
trtpA:avisitn4 0.31098    0.49794   0.625    0.532
```

Nominal variables

GLMM with nominal outcome is created with GLIMMIX (See corresponding SAS section), However, no R functions equivalent to SAS's GLIMMIX procedure have been identified for handling multinomial distributions in a frequentist framwork.

2. SAS GLIMMIX to R

Background

`PROC GLIMMIX` is used in clinical research for generalised linear mixed models, repeated-measures models, non-Gaussian outcomes, random effects, and covariance structures. It supports binary, count, ordinal, and continuous outcomes within one modelling framework.

When moving analyses from SAS to R, the main issue is not just whether the same statistical method exists in R. The main issue is whether the same `PROC GLIMMIX` specification can be translated into an R model with the same outcome distribution, link function, random-effects structure, covariance structure, estimation target, and post-fit inference.

Some `PROC GLIMMIX` models have direct R counterparts. Others require approximation because R packages may not support the same residual covariance structure, likelihood approximation, denominator degrees of freedom method, or `LSMEANS / ILINK` workflow.

Purpose of the Article

This article provides a structured guide for translating common SAS `PROC GLIMMIX` model specifications into R.

The aim is to identify the most suitable R package for each `PROC GLIMMIX` scenario, assess whether the translation is direct or approximate, and document where differences in estimates, standard errors, covariance parameters, or post-fit summaries should be expected.

Scope: Translating `PROC GLIMMIX` Models to R

This article focuses on `PROC GLIMMIX` scenarios where the SAS model includes one or more of the following features:

- non-Gaussian outcomes
- random intercepts or random slopes
- G-side covariance structures
- R-side residual covariance structures
- repeated-measures structures
- overdispersion or zero inflation
- ordinal response models
- spatial or temporal correlation structures
- `LSMEANS`, `ILINK`, odds ratios, and post-fit contrasts

The article does not treat R packages as interchangeable replacements for SAS. Instead, each R implementation is selected according to the `PROC GLIMMIX` model structure, the estimand, and the output required for interpretation.

Comparisons of Languages

PROC GLIMMIX Scenario	Supported in R	Closest R Package	Results Match	Notes
Binary GLMM with random intercept	Yes	`lme4`, `glmmTMB`	Close	Direct GLMM implementation available in both environments
Binary GLMM with random intercept and slope	Yes	`lme4`, `glmmTMB`	Close	Optimisation and covariance parameterisation may differ
Count GLMM with random effects	Yes	`glmmTMB`, `lme4`	Close	Poisson and negative binomial mixed models are supported
Overdispersed count GLMM	Yes	`glmmTMB`	Close	Preferred R route for flexible count dispersion
Ordinal GLMM	Yes	`ordinal`, `glmmTMB`	Close	Package choice depends on link function and random-effects needs
GLIMMIX with structured G-side covariance	Partial	`glmmTMB`, `nlme`	Approximate to expected	Match depends on outcome type and covariance form
GLIMMIX with R-side covariance for binary or count outcomes	Partial	`glmmTMB`, `geepack`	Approximate	Direct residual covariance support is limited for non-Gaussian GLMMs
GLIMMIX with continuous repeated outcomes	Yes	`nlme`, `mmrm`	Close	Strong R support for Gaussian repeated-measures covariance
GLIMMIX post-fit marginal means and contrasts	Yes	`emmeans`	Close	Used for LS-means, response-scale estimates, contrasts, and odds ratios
GLIMMIX used for population-level summaries	Partial	`emmeans`, `geepack`	Approximate	`emmeans` summarises GLMM predictions; `geepack` targets a marginal estimand

Libraries or Extensions Needed

```
library(lme4)
library(glmmTMB)
library(nlme)
library(mmrm)
library(geepack)
library(emmeans)
library(ordinal)
```

The packages above cover the main R translation routes used in this article. `lme4` is used for standard GLMMs, `glmmTMB` for flexible non-Gaussian mixed models, `nlme` and `mmrm` for continuous repeated-measures models, `geepack` for marginal sensitivity analyses, `emmeans` for post-fit inference, and `ordinal` for cumulative link mixed models.

Data Sources for the Analysis

Simulated Data Generation

Synthetic datasets for all scenarios are stored in the GitHub repository:

https://github.com/Franosei/Camis-data-simulation/tree/main/scenario_data

Each scenario folder contains:

- `generate_data.R`: reproducible R data-generation code.
- `generate_data.sas`: SAS code to import the R-generated CSV or create a SAS-native replicate.
- `*.csv`: generated synthetic data for R or SAS analysis.

The commands below are provided for reproducibility. They are not evaluated when this article is rendered because the scenario-generation scripts are stored in the external simulation repository rather than in the CAMIS article directory.

Run all R generators from the repository root:

```
Rscript scenario_data/generate_all_scenarios.R
```

Run one R scenario:

```
Rscript scenario_data/scenario_01_random_intercept/generate_data.R
```

Run all SAS scenario programmes from the repository root:

```
%include "scenario_data/generate_all_scenarios.sas";
```

By default, the SAS programmes import the R-generated CSV files. This gives row-level consistency across R and SAS analyses.

To simulate a SAS-native replicate for one scenario:

```
%let scenario_dir=scenario_data/scenario_01_random_intercept;
%let mode=SIMULATE_IN_SAS;
%include "scenario_data/scenario_01_random_intercept/generate_data.sas";
```

SAS-native simulation uses the same design, seeds, distributions, coefficients, column names, and row counts as the R scripts. Row values may differ because R and SAS use different random-number engines.

Data Preparation Checks Before Model Fitting

Before fitting each model, the analysis data should be prepared consistently in R and SAS. This step is essential because small differences in filtering, factor coding, visit ordering, or derived variables can lead to apparent software differences that are not caused by the modelling procedure itself.

For each scenario, the following checks should be completed before comparing model outputs:

- Apply the same analysis filters in both languages, including `PARAMCD`, population flags, analysis flags, and baseline exclusions.
- Confirm that the same response variable, treatment variable, subject or site identifier, visit variable, and covariates are used.
- Align categorical reference levels for treatment, sex, race, visit, and any other class variables.

- Confirm that binary outcomes use the same event definition in SAS and R.
- Confirm that ordinal outcomes use the same category ordering before comparing cumulative-logit estimates.
- Create derived variables consistently, including centred or scaled covariates, visit factors, and log-offset terms.
- Sort repeated-measures data by subject or site and visit before fitting models with correlation or covariance structures.
- Verify that the number of observations, number of subjects or sites, and maximum records per cluster match across R and SAS.

These checks help ensure that any remaining differences are attributable to software implementation, estimation method, covariance parameterisation, or post-fit inference rather than inconsistencies in the analysis dataset.

Statistical Method

Decision Framework

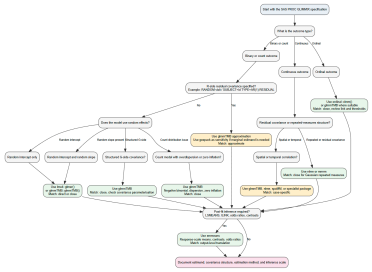
The decision framework starts from the SAS `PROC GLIMMIX` specification and identifies the closest R implementation. The selection is based on outcome type, random-effects structure, covariance structure, distribution, and post-fit inference needs.

How to Read a `PROC GLIMMIX` Specification

Read the SAS model in this order:

1. Identify the response distribution and link function from the `MODEL` statement.
2. Identify the clustering variable from `SUBJECT=`.
3. Check whether the model uses random intercepts, random slopes, or structured G-side covariance.
4. Check whether the `RESIDUAL` option is used, since this indicates R-side covariance.
5. Check whether the analysis requires `LSMEANS`, `ILINK`, odds ratios, or contrasts.
6. Select the R package based on the model feature being translated.

Decision Tree for Selecting the R Implementation



Decision Mapping

PROC GLIMMIX Feature	Outcome Type	SAS Specification Pattern	Closest R Implementation	Match Level	Review Level
Random intercept only	Binary or count	`random intercept / subject=id`	`lme4::glmer()` or `glmmTMB::glmmTMB()`	Direct	Standard
Random intercept and random slope	Binary or count	`random intercept time / subject=id type=un`	`lme4::glmer()` or `glmmTMB::glmmTMB()`	Direct or close	Standard
Structured G-side covariance	Binary, count, or continuous	`random ... / subject=id type=un` or other G-side structure	`glmmTMB`, `nlme`	Close or approximate	Statistical

PROC GLIMMIX Feature	Outcome Type	SAS Specification Pattern	Closest R Implementation	Match Level	Review Level
R-side covariance	Binary or count	`random visit / subject=id type=ar(1) residual`	`glmmTMB` approximation; `geepack` sensitivity	Approximate	SME
R-side covariance	Continuous	`random visit / subject=id type=ar(1) residual`	`nlme::lme()` or `mmrm`	Close	Standard
Count GLMM	Count	`dist=poisson` or `dist=negbin`	`glmmTMB` or `lme4`	Direct or close	Standard
Overdispersed count GLMM	Count	Poisson model with overdispersion or scale adjustment	`glmmTMB`	Close	Statistical
Ordinal GLMM	Ordinal	Cumulative or multinomial ordinal response model	`ordinal::clmm()` or `glmmTMB`	Close	SME
Spatial or temporal correlation	Any	Spatial, temporal, or irregular correlation structure	`glmmTMB`, `nlme`, `spaMM`, or specialist package	Case-specific	SME
Post-fit marginal means	Any	`lsmeans ... / ilink`	`emmeans::emmeans(..., type = "response")`	Output-level match	Output review
Contrasts and odds ratios	Binary or ordinal	`lsmeans ... / diff oddsratio` or `estimate ... / exp`	`emmeans::contrast()` and `emmeans::pairs()`	Output-level match	Output review
Population-level summaries from GLIMMIX	Binary or count	GLIMMIX model with response-scale marginal summaries	`emmeans`; `geepack` as estimand sensitivity	Approximate	SME

Equivalence Criteria for Comparing R and SAS Outputs

The comparison between R and SAS should be interpreted using a hierarchy of equivalence criteria. Exact numerical agreement is not always expected, especially when the software uses different likelihood approximations, covariance parameterisations, denominator degrees of freedom, or post-fit calculation methods.

For each scenario, equivalence is assessed using the following criteria:

- **Model specification:** the same response variable, covariates, treatment groups, reference levels, distribution, link function, clustering variable, and random-effects or covariance structure are used where supported.
- **Analysis population:** the same records are included after applying population flags, analysis flags, parameter filters, visit restrictions, and baseline exclusions.
- **Fixed-effect estimates:** treatment effects and key covariate effects have the same direction, similar magnitude, and comparable standard errors.

- **Covariance parameters:** random-effect variances, standard deviations, correlations, and residual variance estimates are comparable when the same structure is available in both languages.
- **Post-fit estimands:** LS-means, response-scale estimates, odds ratios, rate ratios, treatment differences, and contrasts are compared on the same interpretation scale.
- **Clinical or statistical conclusion:** the overall interpretation remains consistent, even where fit statistics, p-values, or confidence intervals differ slightly.

AIC, BIC, denominator degrees of freedom, and p-values are treated as secondary comparison criteria unless they are central to the analysis objective. Differences in these quantities should be interpreted in the context of the estimation method and software-specific implementation.

Scenario 1: Binary GLMM with Random Intercept

Objective

This scenario translates a SAS `PROC GLIMMIX` binary random-intercept model into R. The outcome is 30-day readmission, coded as `AVAL`, with site-level clustering through `SITEID`.

The model estimates the effect of treatment arm, age, sex, race, comorbidity index, and prior admissions on the odds of readmission. The SAS model uses `PROC GLIMMIX` with a random intercept for site. The R implementation uses `lme4::glmer()` with the same binary logit model and random-intercept structure.

R

```r
library(lme4)
scenario_01 <- read.csv(
  "https://raw.githubusercontent.com/Franosei/Camis-data-
         simulation/main/scenario_data/scenario_01_random_intercept/scenario_01_random_intercept.csv"
)
scenario_01 <- subset(
  scenario_01,
  PARAMCD == "READM30D" &
    ITTFL == "Y" &
    SAFFL == "Y" &
    ANL01FL == "Y"
)
scenario_01$SITEID <- factor(scenario_01$SITEID)
scenario_01$ARMCD <- relevel(
  factor(scenario_01$ARMCD),
  ref = "STDCARE"
)
scenario_01$SEX <- factor(scenario_01$SEX)
scenario_01$RACE <- factor(scenario_01$RACE)
fit_r <- glmer(
  AVAL ~
    ARMCD +
    AGE +
    SEX +
    RACE +
    COMORBIDX +
    PRIORADM +
    (1 | SITEID),
  data = scenario_01,
  family = binomial(link = "logit"),
  control = glmerControl(optimizer = "bobyqa")
)
summary(fit_r)
exp(fixef(fit_r))
```

SAS

```sas
filename scenario temp;
proc http
    url="https://raw.githubusercontent.com/Franosei/Camis-data-
         simulation/main/scenario_data/scenario_01_random_intercept/scenario_01_random_intercept.csv"
    method="GET"
    out=scenario;
run;
proc import datafile=scenario
    out=scenario_01
    dbms=csv
    replace;
```

```
    guessingrows=max;
run;
proc glimmix data=scenario_01 method=laplace;
    where PARAMCD = "READM30D"
        and ITTFL = "Y"
        and SAFFL = "Y"
        and ANL01FL = "Y";
    class SITEID ARMCD(ref="STDCARE") SEX RACE;
    model AVAL(event="1") =
        ARMCD
        AGE
        SEX
        RACE
        COMORBIDX
        PRIORADM
        / dist=binary
          link=logit
          solution
          oddsratio;
    random intercept / subject=SITEID;
run;
```

Results Comparison

Quantity	**R** `lme4::glmer()`	**SAS** `PROC GLIMMIX`
Number of observations	624	624
Number of sites	24	24
Maximum observations per site	34	34
-2 Log Likelihood	747.20	747.18
AIC	767.20	767.18
BIC	811.50	778.96
Random-intercept variance	0.5462	0.5463
Random-intercept standard deviation	0.7391	0.7391

Effect	**R Estimate**	**R Standard Error**	**R p-value**	**SAS Estimate**	**SAS Standard Error**	**SAS p-value**
Intercept	-3.1353	0.6392	<0.0001	-2.4179	0.5865	0.0004
Treatment: care management vs standard care	-0.2968	0.1897	0.1177	-0.2968	0.1902	0.1191
Age	0.0261	0.0081	0.0014	0.0261	0.0082	0.0015
Sex: male vs female	0.2542	0.1856	0.1708	0.2542	0.1860	0.1723
Race: Black or African American vs Asian	0.1248	0.3498	0.7212	0.1248	0.3499	0.7212
Race: Other vs Asian	0.4404	0.4316	0.3076	0.4403	0.4317	0.3079
Race: White vs Asian	0.4636	0.3038	0.1270	0.4636	0.3045	0.1284

Effect	R Estimate	R Standard Error	R p-value	SAS Estimate	SAS Standard Error	SAS p-value
Comorbidity index	0.2853	0.0811	0.0004	0.2854	0.0812	0.0005
Prior admissions	0.2632	0.1281	0.0399	0.2632	0.1283	0.0406

Effect	R Odds Ratio	R 95% CI	SAS Odds Ratio	SAS 95% CI
Treatment: care management vs standard care	0.743	0.512, 1.078	0.743	0.512, 1.080
Age	1.026	1.010, 1.043	1.026	1.010, 1.043
Sex: male vs female	1.289	0.896, 1.855	1.289	0.894, 1.858
Race: Black or African American vs Asian	1.133	0.571, 2.249	1.133	0.570, 2.252
Race: Other vs Asian	1.553	0.667, 3.619	1.553	0.666, 3.622
Race: White vs Asian	1.590	0.876, 2.884	1.590	0.876, 2.884
Comorbidity index	1.330	1.135, 1.559	1.330	1.134, 1.560
Prior admissions	1.301	1.012, 1.672	1.301	1.011, 1.674

Scenario 2: Binary GLMM with Random Intercept and Slope

Objective

This scenario translates a SAS `PROC GLIMMIX` binary random-intercept and random-slope model into R. The outcome is asthma control, coded as `AVAL`, with repeated observations clustered within `USUBJID`.

The model estimates the effect of treatment arm, scaled time, scaled age, sex, race, baseline severity, and scaled adherence on the odds of asthma control. The SAS model uses `PROC GLIMMIX` with independent subject-level random intercept and random slope terms. The R implementation uses `lme4::glmer()` with the matching `||` random-effects structure.

R

```
library(lme4)
scenario_02 <- read.csv(
  "https://raw.githubusercontent.com/Franosei/Camis-data-
        simulation/main/scenario_data/scenario_02_random_intercept_slope/scenario_02_random_intercept_slope.csv"
)
scenario_02 <- subset(
  scenario_02,
  PARAMCD == "ASTCTRL" &
    ITTFL == "Y" &
    SAFFL == "Y" &
    ANL01FL == "Y"
)
```

```r
scenario_02$USUBJID <- factor(scenario_02$USUBJID)
scenario_02$ARMCD <- relevel(
  factor(scenario_02$ARMCD),
  ref = "CTRLTHX"
)
scenario_02$SEX <- factor(scenario_02$SEX)
scenario_02$RACE <- factor(scenario_02$RACE)
scenario_02$TIMEYEAR_C <- as.numeric(
  scale(scenario_02$TIMEYEAR, center = TRUE, scale = TRUE)
)
scenario_02$AGE_C <- as.numeric(
  scale(scenario_02$AGE, center = TRUE, scale = TRUE)
)
scenario_02$ADHERE_C <- as.numeric(
  scale(scenario_02$ADHERE, center = TRUE, scale = TRUE)
)
fit_r <- glmer(
  AVAL ~
    ARMCD +
    TIMEYEAR_C +
    AGE_C +
    SEX +
    RACE +
    BSEVRITY +
    ADHERE_C +
    (1 + TIMEYEAR_C || USUBJID),
  data = scenario_02,
  family = binomial(link = "logit"),
  control = glmerControl(
    optimizer = "bobyqa",
    optCtrl = list(maxfun = 100000)
  )
)
summary(fit_r)
or_ci_r <- exp(
  cbind(
    Odds_Ratio = fixef(fit_r),
    confint(fit_r, parm = names(fixef(fit_r)), method = "Wald")
  )
)
or_ci_r
```

SAS

```sas
```{sas}
{filename scenario temp;}
proc http
    url="https://raw.githubusercontent.com/Franosei/Camis-data-
         simulation/main/scenario_data/scenario_02_random_intercept_slope/scenario_02_random_intercept_slope.csv"
    method="GET"
    out=scenario;
run;
proc import datafile=scenario
    out=scenario_02
    dbms=csv
    replace;
    guessingrows=max;
run;
proc means data=scenario_02 noprint;
    where PARAMCD = "ASTCTRL"
        and ITTFL = "Y"
        and SAFFL = "Y"
        and ANL01FL = "Y";
    var TIMEYEAR AGE ADHERE;
    output out=scales
        mean=mean_TIMEYEAR mean_AGE mean_ADHERE
        std=sd_TIMEYEAR sd_AGE sd_ADHERE;
run;
data scenario_02_scaled;
    if _n_ = 1 then set scales;
    set scenario_02;
    where PARAMCD = "ASTCTRL"
        and ITTFL = "Y"
        and SAFFL = "Y"
        and ANL01FL = "Y";
    TIMEYEAR_C = (TIMEYEAR - mean_TIMEYEAR) / sd_TIMEYEAR;
    AGE_C = (AGE - mean_AGE) / sd_AGE;
    ADHERE_C = (ADHERE - mean_ADHERE) / sd_ADHERE;
```
```

```
run;
proc glimmix data=scenario_02_scaled method=laplace;
    class USUBJID ARMCD(ref="CTRLTHX") SEX RACE;
    model AVAL(event="1") =
        ARMCD
        TIMEYEAR_C
        AGE_C
        SEX
        RACE
        BSEVRITY
        ADHERE_C
        / dist=binary
          link=logit
          solution
          oddsratio;
    random intercept TIMEYEAR_C / subject=USUBJID type=vc;
run;
```

Results Comparison

Quantity	R `lme4::glmer()`	SAS `PROC GLIMMIX`
Number of observations	1400	1400
Number of subjects	280	280
Maximum observations per subject	5	5
-2 Log Likelihood	1670.30	1670.25
AIC	1694.30	1694.25
BIC	1757.20	1737.87
Random-intercept variance	0.6950	0.6949
Random-slope variance for `TIMEYEAR_C`	0.0113	0.0113
Random-intercept standard deviation	0.8337	0.8336
Random-slope standard deviation for `TIMEYEAR_C`	0.1060	0.1062

Effect	R Estimate	R Standard Error	R p-value	SAS Estimate	SAS Standard Error	SAS p-value
Treatment: standard care vs control therapy	-1.1041	0.1888	<0.0001	-1.1042	0.1888	<0.0001
Scaled time	0.5591	0.0671	<0.0001	0.5591	0.0671	<0.0001
Scaled age	0.1601	0.0809	0.0477	0.1601	0.0809	0.0480
Baseline severity	-0.2875	0.1051	0.0062	-0.2875	0.1051	0.0063
Scaled adherence	0.0803	0.0687	0.2421	0.0804	0.0687	0.2422

Effect	R Odds Ratio	R 95% CI	SAS Odds Ratio	SAS 95% CI
Treatment: standard care vs control therapy	0.331	0.229, 0.480	0.331	0.229, 0.480
Scaled time	1.749	1.533, 1.995	1.749	1.533, 1.996
Scaled age	1.174	1.002, 1.375	1.174	1.001, 1.376
Baseline severity	0.750	0.611, 0.922	0.750	0.610, 0.922
Scaled adherence	1.084	0.947, 1.240	1.084	0.947, 1.240